



E4 Educator v2.0

T06

One-wire and analog

Tier 1 · Tutorial 6 of 7

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How one-wire protocol works and why it only needs a single data pin
- How to read the DS18B20 temperature sensor already on the E4 board
- How to address multiple DS18B20 sensors on the same wire
- How the LDR light sensor voltage divider works on the E4
- How to smooth noisy analog readings with averaging
- How to convert raw ADC values into meaningful physical units
- How to build a four-sensor dashboard combining DS18B20, LDR, AHT20, and a bar graph on the OLED

You will need:

- E4 Educator v2.0 board with ESP32, DS18B20, LDR, AHT20, and OLED fitted
- T01 through T05 completed — Button.h available
- USB-C cable (data-capable)
- Optional: second DS18B20 sensor for the multi-sensor exercise

1 The one-wire protocol

One-wire is a serial communication protocol developed by Dallas Semiconductor (now Maxim Integrated) that communicates over a single data wire, shared between the master (ESP32) and one or more slave devices. It earns its name because it only needs one GPIO pin for data — power and ground are separate.

Compared to I2C, one-wire is slower and more timing-sensitive, but it has a unique advantage: you can daisy-chain many sensors on the same single wire and address each one individually by its unique 64-bit ROM code burned in at the factory.

1.1 One-wire on the E4

Element	E4 detail	Notes
Data pin	GPIO 4	ONE_WIRE define from build_flags. This is the single data line.
Pull-up resistor	4.7kΩ fitted	Already on the E4 board between GPIO 4 and 3V3. Required by the one-wire spec — do not omit.
Sensor fitted	DS18B20	Waterproof probe version on the E4. Address discovered at runtime.
Library	OneWire + DallasTemperature	Two libraries work together. OneWire handles the protocol, DallasTemperature handles DS18B20 specifics.

1.2 The 64-bit ROM address

Every DS18B20 sensor has a unique 64-bit address burned in at manufacture. This allows multiple sensors on the same wire to be addressed individually. In practice you have two choices:

- Let the library find all sensors and address them by index (position on the bus) — simple but order can change if sensors are added or removed
- Read each sensor's ROM address once and store it — robust but requires a discovery step first

For the E4 course, the index approach is used for simplicity. The ROM address approach is covered in the multi-sensor exercise.

2 Reading the DS18B20

2.1 Install the libraries

```
[env:e4_educator_v2]
; ... existing settings ...
lib_deps =
  paulstoffregen/OneWire
  milesburton/DallasTemperature
  adafruit/Adafruit AHTX0
  adafruit/Adafruit SSD1306
  adafruit/Adafruit GFX Library
  adafruit/Adafruit BusIO
```

2.2 Basic DS18B20 reading

```
#include <Arduino.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include "Button.h"

OneWire      oneWire(ONE_WIRE);
DallasTemperature ds(&oneWire);
Button       nextBtn(BTN_NEXT);

unsigned long lastRead    = 0;
const unsigned long READ_INTERVAL = 2000;

void setup() {
  Serial.begin(115200);
  Serial.printf("T06 - DS18B20 Reader  FW: %s\n", FW_VERSION);

  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED, OUTPUT);
  digitalWrite(LED_GREEN, LOW);
  digitalWrite(LED_RED, LOW);

  nextBtn.begin();
  ds.begin();

  int count = ds.getDeviceCount();
  Serial.printf("DS18B20 sensors found: %d\n", count);

  if (count == 0) {
    Serial.println("ERROR: No DS18B20 found. Check ONE_WIRE pin.");
    digitalWrite(LED_RED, HIGH);
    while (true) { delay(100); }
  }

  // Set resolution: 9=0.5C, 10=0.25C, 11=0.125C, 12=0.0625C
  ds.setResolution(12); // Highest resolution, ~750ms conversion
  ds.setWaitForConversion(false); // Non-blocking conversion
```

```

    digitalWrite(LED_GREEN, HIGH);
    Serial.println("Ready. Press NEXT for immediate reading.");
}

void loop() {
    unsigned long now = millis();
    Button::Event nextEv = nextBtn.read();

    bool shouldRead = (now - lastRead >= READ_INTERVAL);
    if (nextEv == Button::SHORT_PRESS) shouldRead = true;

    if (shouldRead) {
        lastRead = now;

        // Request conversion (non-blocking)
        ds.requestTemperatures();

        // Wait for conversion to complete (750ms at 12-bit)
        // In a real project use millis() here too – see Section 2.3
        delay(750);

        float tempC = ds.getTempCByIndex(0);

        if (tempC == DEVICE_DISCONNECTED_C) {
            Serial.println("ERROR: Sensor disconnected");
            return;
        }

        Serial.printf("[%lu ms] DS18B20: %.2f C\n", now, tempC);
    }
}

```

The 750ms conversion time

At 12-bit resolution the DS18B20 takes up to 750ms to convert a temperature. The code above uses `delay(750)` for simplicity. Section 2.3 shows the correct `millis()` pattern for production firmware where you cannot afford to block for 750ms.

2.3 Non-blocking DS18B20 conversion

The correct pattern uses `millis()` to wait for the conversion without blocking:

```

// State machine for non-blocking DS18B20 reading
enum DS18State { DS18_IDLE, DS18_CONVERTING };
DS18State ds18State = DS18_IDLE;

unsigned long conversionStarted = 0;
const unsigned long CONVERSION_TIME = 750; // ms at 12-bit

float ds18TempC = 0.0;

```

```
void updateDS18B20(unsigned long now) {
  switch (ds18State) {
    case DS18_IDLE:
      // Start a new conversion
      ds.requestTemperatures();
      conversionStarted = now;
      ds18State = DS18_CONVERTING;
      break;

    case DS18_CONVERTING:
      // Wait without blocking
      if (now - conversionStarted >= CONVERSION_TIME) {
        ds18TempC = ds.getTempCByIndex(0);
        ds18State = DS18_IDLE;
        if (ds18TempC != DEVICE_DISCONNECTED_C) {
          Serial.printf("DS18B20: %.2f C\n", ds18TempC);
        }
      }
      break;
  }
}

// In loop():
// Call updateDS18B20(now) every loop pass.
// It manages its own state and never blocks.
```

★ Key point

The two-state DS18B20 machine — IDLE (request conversion) and CONVERTING (wait then read) — is the correct non-blocking pattern. It uses only 3 lines of state and costs nothing while waiting. This is exactly the kind of state machine thinking that separates embedded systems developers from Arduino sketch writers.

3 Multiple DS18B20 sensors on one wire

One of the DS18B20's most useful features is that many sensors can share the same single wire. Each is individually addressable by its ROM code. This is extremely practical for temperature measurement across multiple points — a weather station, a brewing setup, multiple rooms.

3.1 Discover ROM addresses

Run this sketch with multiple DS18B20 sensors connected to the ONE_WIRE pin. It prints the ROM address of every sensor found — copy these addresses into your main firmware to address each sensor by name rather than by index.

```
#include <Arduino.h>
#include <OneWire.h>
#include <DallasTemperature.h>

OneWire      oneWire(ONE_WIRE);
DallasTemperature ds(&oneWire);

void setup() {
  Serial.begin(115200);
  ds.begin();

  int count = ds.getDeviceCount();
  Serial.printf("Found %d DS18B20 sensor(s)\n\n", count);

  for (int i = 0; i < count; i++) {
    DeviceAddress addr;
    if (ds.getAddress(addr, i)) {
      Serial.printf("Sensor %d ROM: {", i);
      for (int b = 0; b < 8; b++) {
        Serial.printf("0x%02X", addr[b]);
        if (b < 7) Serial.print(", ");
      }
      Serial.println("};");
    }
  }
}

void loop() {}
```

This outputs something like:

```
Found 2 DS18B20 sensor(s)

Sensor 0 ROM: {0x28, 0xFF, 0x4A, 0x1B, 0x32, 0x18, 0x01, 0xC8};
Sensor 1 ROM: {0x28, 0xFF, 0x91, 0x3C, 0x32, 0x18, 0x01, 0x5F};
```

Copy these addresses into your firmware as named constants:

```
// Named sensor addresses — paste from discovery sketch
```

```
DeviceAddress sensorInternal = {0x28, 0xFF, 0x4A, 0x1B, 0x32, 0x18, 0x01, 0xC8};  
DeviceAddress sensorExternal = {0x28, 0xFF, 0x91, 0x3C, 0x32, 0x18, 0x01, 0x5F};  
  
// Read by name instead of by index  
float tempInternal = ds.getTempC(sensorInternal);  
float tempExternal = ds.getTempC(sensorExternal);  
  
Serial.printf("Internal: %.2f C External: %.2f C\n",  
              tempInternal, tempExternal);
```

★ Key point

Addressing by ROM code instead of by index makes your firmware robust — sensors can be added, removed, or reordered on the bus without changing which reading maps to which name. This is the production-quality approach for multi-sensor deployments.

4 The LDR light sensor

An LDR (Light Dependent Resistor) changes its resistance based on the amount of light falling on it. In bright light its resistance drops. In darkness its resistance rises. The E4 uses the LDR in a voltage divider circuit to convert this resistance change into a voltage that the ADC can measure.

4.1 How the voltage divider works

The E4 LDR circuit connects the LDR in series with a fixed resistor (22kΩ) between 3V3 and GND. The midpoint — where LDR meets the fixed resistor — connects to GPIO 39 (LDR define).

As light increases, LDR resistance drops, the voltage at the midpoint rises, and the ADC reads a higher value. In darkness, LDR resistance is high, the voltage drops, and the ADC reads a lower value.

Condition	ADC reading	Meaning
Bright light	~3500–4095	LDR resistance low, voltage at midpoint high
Indoor light	~1500–2500	Typical office or room lighting
Covered / dark	~0–500	LDR resistance high, voltage at midpoint low

ADC1 rule reminder

GPIO 39 (LDR) is an ADC1 pin — safe to use with WiFi active. Never attempt to use ADC2 pins for the LDR or any analog sensor when WiFi is enabled. The E4 v2.0 correctly assigns all analog sensors to ADC1 pins.

4.2 Smoothing noisy readings with averaging

ADC readings on the ESP32 contain noise — even in a stable lighting environment, consecutive readings vary by tens of counts. For smooth, reliable output, average multiple readings:

```
// Read LDR with averaging to reduce noise
int readLDRSmoothed(int samples = 8) {
    long total = 0;
    for (int i = 0; i < samples; i++) {
        total += analogRead(LDR);
        delay(2); // Short delay between samples
    }
    return total / samples;
}

// Convert raw ADC to a 0-100 light percentage
int rawToPercent(int raw) {
    int pct = map(raw, 200, 3800, 0, 100);
    return constrain(pct, 0, 100);
}

// Usage in loop():
int rawLDR = readLDRSmoothed(8);
int lightPct = rawToPercent(rawLDR);
Serial.printf("LDR raw: %4d  Light: %3d%%\n", rawLDR, lightPct);
```


★ Key point

Averaging 8 samples reduces noise significantly and costs only 16ms — well within the tolerance of any sensor reading loop. For fast-moving signals like audio, averaging would blur the signal — but for slow-changing environmental measurements like light level, averaging is always the right choice.

5 Drawing a bar graph on the OLED

A bar graph is a natural way to visualise the LDR light level. It uses the drawing primitives from T05 to create a dynamic visual indicator that updates with each reading.

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_WIDTH 128
#define OLED_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDR 0x3C

Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, OLED_RESET);

// — Bar graph zone —————
#define BAR_X      0
#define BAR_Y      48
#define BAR_W      128
#define BAR_H      12
#define BAR_BORDER  1

void drawBarGraph(int percent, const char* label) {
  // Erase bar zone
  oled.fillRect(BAR_X, BAR_Y - 10, BAR_W, BAR_H + 10, SSD1306_BLACK);

  // Label
  oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(BAR_X, BAR_Y - 9);
  oled.printf("%s %d%", label, percent);

  // Bar outline
  oled.drawRect(BAR_X, BAR_Y, BAR_W, BAR_H, SSD1306_WHITE);

  // Bar fill — map percent to pixel width inside border
  int fillW = map(percent, 0, 100, 0, BAR_W - (BAR_BORDER * 2));
  fillW = constrain(fillW, 0, BAR_W - (BAR_BORDER * 2));
  if (fillW > 0) {
    oled.fillRect(BAR_X + BAR_BORDER,
                  BAR_Y + BAR_BORDER,
                  fillW,
                  BAR_H - (BAR_BORDER * 2),
                  SSD1306_WHITE);
  }

  oled.display();
}

// Call with a 0-100 percent value:
// drawBarGraph(lightPct, "Light");
```

6 Practical exercise — four-sensor dashboard

This exercise brings together all the sensors on the E4 into a single dashboard project. Two OLED pages show different combinations of data. The DS18B20 uses the non-blocking state machine from Section 2.3.

Page 1: DS18B20 temperature + LDR bar graph. Page 2: AHT20 temperature and humidity. NEXT short press cycles pages. NEXT long press forces an immediate sensor read on all sensors.

```
#include <Arduino.h>
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Adafruit_AHTX0.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "Button.h"

// — Hardware —————
#define OLED_WIDTH 128
#define OLED_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDR 0x3C

Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, OLED_RESET);
Adafruit_AHTX0 aht;
OneWire oneWire(ONE_WIRE);
DallasTemperature ds(&oneWire);
Button nextBtn(BTN_NEXT);

// — State —————
enum Page { PAGE_DS18_LDR, PAGE_AHT20, PAGE_COUNT };
enum DS18St { DS18_IDLE, DS18_CONVERTING };

Page currentPage = PAGE_DS18_LDR;
DS18St ds18State = DS18_IDLE;

unsigned long lastSensorRead = 0;
unsigned long conversionStart = 0;
const unsigned long SENSOR_INTERVAL = 3000;
const unsigned long CONVERSION_TIME = 750;

float ds18Temp = 0.0;
float ahtTemp = 0.0;
float ahtHumid = 0.0;
int lightPct = 0;

// — LDR —————
int readLDR() {
    long total = 0;
    for (int i = 0; i < 8; i++) { total += analogRead(LDR); delay(2); }
    return constrain(map(total / 8, 200, 3800, 0, 100), 0, 100);
}
```

```

// — DS18B20 non-blocking state machine —————
void updatedDS18(unsigned long now) {
    switch (ds18State) {
        case DS18_IDLE:
            ds.requestTemperatures();
            conversionStart = now;
            ds18State = DS18_CONVERTING;
            break;
        case DS18_CONVERTING:
            if (now - conversionStart >= CONVERSION_TIME) {
                float t = ds.getTempCByIndex(0);
                if (t != DEVICE_DISCONNECTED_C) ds18Temp = t;
                ds18State = DS18_IDLE;
            }
            break;
    }
}

// — AHT20 read —————
void readAHT20() {
    sensors_event_t humidity, temp;
    aht.getEvent(&humidity, &temp);
    ahtTemp = temp.temperature;
    ahtHumid = humidity.relative_humidity;
}

// — OLED pages —————
void drawPageDS18LDR() {
    oled.clearDisplay();
    oled.fillRect(0, 0, 128, 14, SSD1306_WHITE);
    oled.setTextColor(SSD1306_BLACK);
    oled.setTextSize(1);
    oled.setCursor(2, 3);
    oled.print("Probe + Light 1/2");
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(2);
    oled.setCursor(0, 18);
    if (ds18Temp != 0.0)
        oled.printf("%.2f C", ds18Temp);
    else
        oled.print("--.- C");
    // Light bar
    oled.setTextSize(1);
    oled.setCursor(0, 41);
    oled.printf("Light %3d%%", lightPct);
    oled.drawRect(0, 52, 128, 10, SSD1306_WHITE);
    int fw = map(lightPct, 0, 100, 0, 126);
    if (fw > 0) oled.fillRect(1, 53, fw, 8, SSD1306_WHITE);
    oled.display();
}

void drawPageAHT20() {
    oled.clearDisplay();
    oled.fillRect(0, 0, 128, 14, SSD1306_WHITE);

```

```

    oled.setTextCursor(2, 3);
    oled.print("Temp+Humidity 2/2");
    oled.setTextCursor(0, 18);
    oled.printf("%.1f C", ahtTemp);
    oled.setTextCursor(0, 42);
    oled.printf("%.1f %%", ahtHumid);
    oled.display();
}

void refreshPage() {
    switch (currentPage) {
        case PAGE_DS18_LDR: drawPageDS18LDR(); break;
        case PAGE_AHT20:    drawPageAHT20();    break;
        default: break;
    }
}

// — Setup —————
void setup() {
    Serial.begin(115200);
    Serial.printf("T06 Exercise - 4-Sensor Dashboard FW: %s\n", FW_VERSION);

    Wire.begin(I2C_SDA, I2C_SCL);

    pinMode(LED_GREEN, OUTPUT);
    pinMode(LED_RED,   OUTPUT);
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_RED,   LOW);

    nextBtn.begin();

    bool ok = true;
    if (!oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
        Serial.println("ERROR: OLED not found"); ok = false;
    }
    if (!aht.begin(&Wire)) {
        Serial.println("ERROR: AHT20 not found"); ok = false;
    }
    ds.begin();
    ds.setResolution(12);
    ds.setWaitForConversion(false);
    if (ds.getDeviceCount() == 0) {
        Serial.println("ERROR: DS18B20 not found"); ok = false;
    }
    if (!ok) { digitalWrite(LED_RED, HIGH); while(true) { delay(100); } }

    digitalWrite(LED_GREEN, HIGH);
    readAHT20();
    lightPct = readLDR();
    refreshPage();
}

```

```

    Serial.println("Ready. NEXT=page  NEXT long=force read");
}

// — Loop —————
void loop() {
    unsigned long now = millis();
    Button::Event nextEv = nextBtn.read();

    // NEXT short: cycle page
    if (nextEv == Button::SHORT_PRESS) {
        currentPage = (Page)((currentPage + 1) % PAGE_COUNT);
        refreshPage();
    }

    // NEXT long: force immediate read of all sensors
    if (nextEv == Button::LONG_PRESS) {
        readAHT20();
        lightPct = readLDR();
        ds18State = DS18_IDLE; // Restart DS18B20 conversion
        lastSensorRead = now;
        Serial.println("Forced read.");
    }

    // DS18B20 non-blocking state machine — runs every loop
    updateDS18(now);

    // Timed sensor read — AHT20 and LDR
    if (now - lastSensorRead >= SENSOR_INTERVAL) {
        lastSensorRead = now;
        readAHT20();
        lightPct = readLDR();
        refreshPage();
        Serial.printf("DS18:%.2fC  AHT:%.1fC  Hum:%.1f%%  Light:%d%%\n",
                      ds18Temp, ahtTemp, ahtHumid, lightPct);
    }

    // Refresh page when DS18B20 conversion completes
    static DS18St prevDS18State = DS18_IDLE;
    if (prevDS18State == DS18_CONVERTING && ds18State == DS18_IDLE) {
        refreshPage();
    }
    prevDS18State = ds18State;
}

```

6.1 What to observe

- Page 1 shows the DS18B20 probe temperature with a live LDR bar graph — cover the LDR and watch the bar shrink
- Page 2 shows AHT20 temperature and humidity — breathe on the sensor to see humidity rise
- NEXT short press cycles between pages instantly with no delay
- NEXT long press forces an immediate read of all sensors and updates the current page
- The DS18B20 reading updates asynchronously — the page refreshes when the 750ms conversion completes without blocking anything

- All four sensors report to Serial simultaneously every 3 seconds

Try this

Hold the DS18B20 probe between your fingers for 30 seconds and watch the temperature climb on page 1. Then switch to page 2 and compare the AHT20 temperature reading at the same time — two independent temperature measurements of different things showing the value of multiple sensors.

7 T06 summary

Concept	Key takeaway
One-wire protocol	Single data pin, 4.7kΩ pull-up required, multiple sensors addressable by 64-bit ROM code. E4 pull-up already fitted.
DS18B20 resolution	12-bit = 0.0625°C resolution, 750ms conversion. Use <code>setWaitForConversion(false)</code> and a state machine for non-blocking reads.
ROM addressing	Use discovery sketch to find addresses. Address sensors by ROM code not by index for robust multi-sensor firmware.
LDR voltage divider	LDR in series with 22kΩ to GND. Midpoint to GPIO 39 (ADC1). Higher light = higher ADC reading on E4.
ADC averaging	Average 8 samples to reduce noise. 16ms cost. Always use for slow-changing environmental measurements.
Bar graph	<code>drawRect()</code> for outline, <code>fillRect()</code> for fill scaled with <code>map()</code> . Erase zone with BLACK before redrawing.
DS18B20 state machine	DS18_IDLE (request conversion) → DS18_CONVERTING (wait 750ms) → DS18_IDLE (read value). Zero blocking.

What's next

- T07 — Audio output basics: the final Tier 1 tutorial covers piezo tones with LEDC, DAC sine wave audio, and PWM transistor load control — rounding out the full E4 hardware toolkit before the big step up to the TFT display in Tier 2.